

Energy Efficient Task Offloading and Resource Allocation in Air-Ground Integrated MEC Systems: A Distributed Online Approach

Ying Chen[✉], Senior Member, IEEE, Kaixin Li[✉], Yuan Wu[✉], Senior Member, IEEE,
Jiwei Huang[✉], Senior Member, IEEE, and Lian Zhao[✉], Fellow, IEEE

Abstract—In many remote areas lacking ground communication infrastructure support, such as wilderness, desert, ocean, etc., an integrated edge computing network in the air with edge computing nodes is an effective solution. It can provide over-the-air computing services for ground devices (GDs) with limited computing resources and battery life. In this paper, we study task offloading and resource allocation in the aerial-based mobile edge computing (MEC) system supported by a high altitude platform (HAP) and unmanned aerial vehicles (UAVs), with the goal of minimizing the GD's energy consumption. Considering that the task arrival of GDs and wireless communication quality are both stochastic and dynamic, we apply stochastic optimization techniques to transform this task offloading and resource allocation problem into two subproblems, i.e., 1) a subproblem for local computation resource allocation and 2) a subproblem for offloading resource allocation. For the first subproblem, we use convex optimization methods to address it. For the second subproblem, we use game theory to formulate the competition of offloading resources among GDs and propose the Distributed Game-theoretical Multi-server Selection (DGMS) algorithm and the Transmission Power Allocation (TPA) algorithm. Finally, we propose a Distributed Online Task Offloading and Resource Allocation (DOTORA) algorithm and give the theoretical performance analysis of the algorithm. We perform extensive experiments, including the comparison experiments with the UAV-Only and HAP-Only framework, and the comparison experiments with other algorithms under our HAP-UAV framework. The experimental results validate our proposed framework and the DOTORA algorithm.

Index Terms—Air-ground integrated networks, game theory, mobile edge computing (MEC), resource allocation, task offloading.

I. INTRODUCTION

WITH the rapid development of wireless networks and mobile communication technologies, a variety of new network applications have emerged, such as autonomous driving, face recognition, and so on [1], [2]. These applications often have stringent requirements for latency and battery energy, while ground devices (GDs) have difficulty handling these tasks efficiently because of limited computing power and battery life [3], [4]. Therefore, mobile edge computing (MEC) has been proposed to address the above issues [5], [6]. MEC deploys more resourceful computing facilities at the edge of the network, which can effectively increase the communication and computing power of the network [7]. GDs can effectively reduce energy consumed by offloading computing tasks to edge servers for proximity processing via wireless access points [8].

Traditional MEC deployed on wireless cellular networks usually relies on existing infrastructure and communication facilities on the ground. However, in some remote areas, such as wilderness, deserts, and oceans, terrestrial infrastructure communication facilities may be lacking or damaged, making the MEC network unreachable. Besides, the ground-based MEC system can be severely damaged in the event of a natural disaster [9]. In these cases, it is hard to establish a communication system based on a terrestrial environment, and the cost of establishing a terrestrial environment communication system is also very expensive [10]. Unmanned aerial vehicles (UAVs) with good maneuverability and flexibility can be applied in these scenarios [11]. Therefore, the UAV-assisted MEC framework has become a research hotspot [12], [13]. It can help ground users handle many computing tasks by using edge servers equipped with computing resources, significantly improving the quality of user experience [14].

Although UAV-assisted MEC networks have many advantages, the limited computing resources and battery life of UAVs make it difficult to meet the demands of a large number of intensive computing tasks and may not consistently provide computing services to GDs. To solve this problem, high altitude platforms (HAPs) are integrated as a complement to make up for UAV limitations [15]. HAP operates at high altitudes for a long

Manuscript received 3 September 2023; revised 24 November 2023; accepted 11 December 2023. Date of publication 25 December 2023; date of current version 2 July 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61972414, in part by Beijing Natural Science Foundation under Grant L232050, in part by the Project of Cultivation for young top-match Talents of Beijing Municipal Institutions under Grant BPHR202203225, in part by Young Elite Scientists Sponsorship Program by BAST under Grant BYESS2023031, in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2022A1515011287, and in part by MYRG2020-00107-IOTSC. Recommended for acceptance by D. Niyato. (Corresponding authors: Yuan Wu; Jiwei Huang.)

Ying Chen and Kaixin Li are with Beijing Information Science and Technology University, Beijing 100101, China (e-mail: chenying@bistu.edu.cn; likaixin@bistu.edu.cn).

Yuan Wu is with the State Key Lab of Internet of Things for Smart City, University of Macau, Macao 999078, China, and also with the Department of Computer and Information Science, University of Macau, Macao 999078, China (e-mail: yuanwu@um.edu.mo).

Jiwei Huang is with the Beijing Key Laboratory of Petroleum Data Mining, China University of Petroleum, Beijing 102249, China (e-mail: huangjiw@cup.edu.cn).

Lian Zhao is with Toronto Metropolitan University, Toronto, ON M5B 2K3, Canada (e-mail: l5zhao@torontomu.ca).

Digital Object Identifier 10.1109/TMC.2023.3346431

time and is not affected by ground weather. Besides, HAP can cover a large ground area and provide stable communication services, while UAVs are limited by their flight heights and generally can only operate in local areas. Therefore, it is important for UAVs and HAPs to cooperate to provide computing services for GDs [16]. Without relying on fixed infrastructure, HAP and UAV can be deployed quickly and flexibly according to actual needs and are adapted to different environments and scenarios. In addition, HAP and UAV can realize data transmission and processing through air links, forming a distributed edge computing network, and improving computing efficiency and load balancing. Therefore, the task offloading and resource allocation in the aerial-based MEC system of HAP and UAV cooperation has garnered increasing attention from academia and industry [17].

However, solving the above problems faces several challenges [18]. First, the generation and arrival of computing tasks of GDs are dynamic, and the communication quality of wireless channels is also constantly changing. At the same time, task arrival and channel quality are also subject to numerous external environments. Therefore, it is difficult to accurately obtain the statistical information in advance. Second, the computing and communication resources of HAPs and UAVs are still limited and the resource costs are high. Each GD has to compete for limited computing and communication resources to achieve a satisfactory experience. Therefore, it is a challenge to find the balanced task offloading and resource allocation strategy while minimizing the energy consumption of GDs.

In this paper, we study the distributed online task offloading and resource allocation in the aerial-based MEC system. Our goal is to minimize the GD's total energy consumption while guaranteeing the performance and satisfying the constraints of offloading resource constraints. The control decisions include local computation resource allocation, server selection, and transmission power allocation. This problem is difficult to solve directly because of the coupling of these control decisions. To this end, we employ stochastic optimization techniques to decompose the problem into two subproblems: (1) a subproblem for local computation resource allocation, and (2) a subproblem for offloading resource allocation. We develop corresponding solutions for each subproblem and propose a Distributed Online Task Offloading and Resource Allocation (DOTORA) algorithm. Finally, we validate the performance of the DOTORA algorithm through theoretical analysis and a series of experiments. The key contributions of this paper are summarized as follows:

- 1) We investigate the problem of task offloading and resource allocation in the aerial-based MEC system, where HAP and UAVs are deployed with edge servers. Tasks for GDs can be either processed locally or offloaded to edge servers on UAVs or the HAP. Our goal is to minimize the GD's energy consumption. Considering the significant features of the time-varying network, we simultaneously consider local computation resource allocation, server selection, and transmission power allocation decisions.
- 2) Considering that the task generation and arrival of GDs are both dynamic and stochastic, and the network communication environment is time-varying, we use stochastic

optimization techniques to transform the above problem and decompose the problem into two subproblems. The first subproblem is local computation resource allocation, which is solved using a convex optimization method. The second subproblem is offloading resource allocation. We take advantage of game theory to reformulate the subproblem as the game-theoretical multi-server selection (GMS) game, propose the corresponding potential function, and prove theoretically that the GMS game is a potential game. Then, we propose a Distributed Game-theoretical Multi-server Selection (DGMS) algorithm and a Transmission Power Allocation (TPA) algorithm to solve the second subproblem.

- 3) We propose the DOTORA algorithm to effectively solve the above task offloading and resource allocation problems. The algorithm can be implemented in a distributed manner. In addition, we provide a rigorous theoretical analysis of the performance of the proposed algorithm.
- 4) We conduct extensive experiments to evaluate the DOTORA algorithm. A series of parameter analysis experiments show that the DOTORA algorithm can achieve a flexible trade-off between energy consumption and performance. Then, we compare the HAP-UAV framework with the UAV-Only and HAP-Only frameworks, which shows the superiority of the integrated HAP-UAV frameworks. In addition, we compare the DOTORA algorithm with other task offloading and resource allocation methods, and the results indicate that our algorithm can significantly reduce the energy consumption of GDs while guaranteeing the performance.

The remainder of this paper is organized as follows. Section II establishes the system model and formulates the task offloading and resource allocation problem. Section III reformulates the problem using stochastic optimization techniques and proposes the DOTORA algorithm. Section IV provides the theoretical analysis of DOTORA's performance. Section V gives the experimental evaluation. The related work is presented in Section VI. Finally, Section VII concludes this paper.

II. SYSTEM MODEL AND PROBLEM FORMULATION

A. Aerial-Based MEC Framework

Considering an aerial-based MEC framework shown in Fig. 1, which comprises an air computing layer containing HAP and UAV, and a ground layer containing various GDs. The ground layer consists of GDs with limited computing resources and battery capacity, and the set is designated as $\mathcal{N} = \{1, 2, \dots, n, \dots, N\}$. The aerial layer is composed of a HAP and UAVs that are equipped with edge servers, and the set of servers is designated as $\mathcal{S} = \{0, 1, \dots, s, \dots, S\}$. Therefore, there are $S + 1$ servers available to serve the GD. Among them, $s = 0$ means the server is connected with HAP, and other means the servers are connected with UAVs. Typically, GDs produce a significant quantity of computation-intensive tasks, while their local computing resources are limited. In this framework, GDs can offload part of the tasks to HAP or UAVs for processing. Tasks arrive at the GD in a stochastic fashion during each time slot, measured by data bits, and the tasks are separable. In

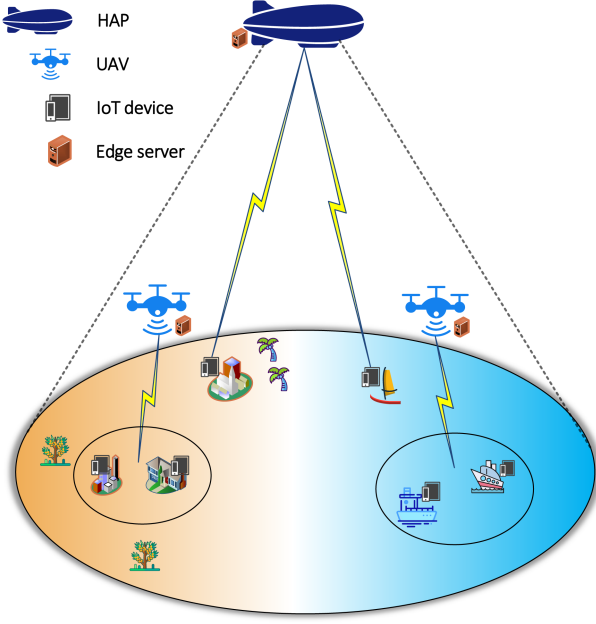


Fig. 1. Aerial-based MEC framework.

addition, GD can offload tasks directly to HAP for processing to reduce waiting latency when the UAV is inadequate to handle tasks from the GD. In this system, time is discretized into a slot model, as $\mathcal{T} = \{0, 1, \dots, t, \dots, T-1\}$. Wherein, the duration of each time slot is τ . Each GD competes with other GDs for limited communication resources in order to obtain a desirable user experience. For the convenience of readers' reference, the main symbols are listed in Table I.

B. Communication Model

The server selection decision for GD n is defined as $v_n^s(t)$, where $v_n^s(t) = 1$ indicates that GD n selects server s to handle tasks in t th slot, or else $v_n^s(t) = 0$. $\mathbf{v}(t) = \{v_n^s(t) | n \in \mathcal{N}\}$ indicate the server selection decision of all GDs in t th slot. In addition, the transmission power of GD n during communication with server is represented as $p_n(t)$, and $0 \leq p_n(t) \leq p_n^{\max}$.

1) *Ground-HAP Communication Model*: In the Ground-HAP communication model, GDs communicate with the HAP, where the HAP hovers at a fixed height in the stratosphere. According to [19], the path loss when the GD n communicates with the HAP is calculated as

$$L_n = 20 \log_{10} \left(\frac{4\pi f_c \sqrt{d_n^2 + r_n^2}}{c} \right) + \rho_n^{LoS} \eta_n^{LoS} + (1 - \rho_n^{LoS}) \eta_n^{NLoS}, \quad (1)$$

where d_n represents the vertical distance from the HAP to the ground, r_n indicates the horizontal distance from GD n to the HAP, f_c denotes the carrier frequency, and c denotes the speed of light. Furthermore, η_n^{LoS} and η_n^{NLoS} denote line-of-sight (LoS) and non-LoS (NLoS) link loss when GD n communicates with the HAP, respectively. Besides, the LoS communication

TABLE I
KEY NOTATIONS

Notations	Definitions
$\mathcal{N}$	the set of GDs
$\mathcal{S}$	the set of edge server
τ	the length of time slot
η_n^{LoS}	the line-of-sight link loss
η_n^{NLoS}	the non-LoS link loss
ρ_n^{LoS}	the LoS communication probability of GD n
$v_n^s(t)$	the server selection decision of GD n
$\mathbf{v}(t)$	the server selection decision of all GDs
$f_n^l(t)$	the CPU cycle frequency of GD n
q_n	the CPU cycles needed by GD n to process each bit of data
$p_n(t)$	the transmission power decision of GD n
$\mathbf{p}(t)$	the transmission power decision of all GDs
$g_n^s(t)$	the channel gain when GD n communicates with HAP or UAV
$\gamma_n^s(t)$	the signal-to-noise ratio when GD n communicates with HAP or UAV
$R_n^s(t)$	the data transmission rate of GD n communicates with HAP or UAV
$A_n(t)$	the size of tasks that are newly arriving at GD n
$W_n^l(t)$	the size of task processed locally by the GD n
$W_n^o(t)$	the size of task offloaded by GD n
$G_n(t)$	the queue backlog of the buffer queue of GD n
$E_n^l(t)$	the energy consumed by local computation of GD n
$E_n^o(t)$	the energy consumed by GD n offloading tasks

probability is calculated as

$$\rho_n^{LoS} = \frac{1}{1 + \kappa_1 \exp\{-\kappa_2 [\tan^{-1}(\frac{d_n}{r_n}) - \kappa_1]\}}, \quad (2)$$

where the value of $\kappa_1, \kappa_2, \eta_n^{LoS}$ and η_n^{NLoS} is determined by the state of the environment.

Without loss of generality, the channel gain when GD n communicates with the HAP is

$$g_n^s(t) = 10^{-\frac{L_n}{10}}, s = 0. \quad (3)$$

2) *Ground-UAV Communication Model*: In this model, the communication distance between GD n and UAV s is l_n^s . According to [20], the channel gain when GD n communicates with UAV s is denoted as

$$g_n^s(t) = 128 + 37.6 \log_{10}(l_n^s), s = 1, \dots, S. \quad (4)$$

In summary, the communication interference from other GDs when GD n offloads its task to server s is $\sum_{m \neq n} v_m^s(t) p_m(t) g_m^s(t)$. Therefore, the signal-to-noise ratio when GD n communicates with server s is calculated as

$$\gamma_n^s(t) = \frac{v_n^s(t) p_n(t) g_n^s(t)}{\sum_{m \neq n} v_m^s(t) p_m(t) g_m^s(t) + \sigma^2}, \quad (5)$$

where $\sum_{s=0}^S v_n^s(t) \leq 1, \forall n \in \mathcal{N}$ and σ^2 stands for the noise power.

Therefore, the data transmission rate of GD n is denoted as

$$R_n^s(t) = B_s \log_2(1 + \gamma_n^s(t)), \quad (6)$$

where B_s is the bandwidth allocated by server s .

C. Task and Queue Model

In our system model, a buffer queue is maintained by each GD to store the newly arrived tasks. The task processed by the GD n locally is $W_n^l(t)$ (in bits), indicated as

$$W_n^l(t) = \frac{f_n^l(t)\tau}{\varrho_n}, \quad (7)$$

where $f_n^l(t)$ is the CPU frequency of GD n , which satisfies $0 \leq f_n^l(t) \leq f_n^{\max}$, and ϱ_n represents the CPU cycles needed by GD n to process each bit of data.

Each GD can only choose one server to offload tasks within each slot, and the choice may be different within different slots. Define the size of the computation offloaded to the server s is $W_n^o(t)$ (in bits), expressed by

$$W_n^o(t) = \sum_{s=0}^S R_n^s(t)\tau. \quad (8)$$

The size of tasks that are newly arriving at GD n in t th slot is symbolized by $A_n(t)$ (in bits), and $A_n(t) \leq A_n^{\max}(t)$. Simultaneously, define the queue backlog of the buffer queue of GD n to be $G_n(t)$. After a time slot ends, the size of buffer queue will change, and this change process is described as

$$G_n(t+1) = [G_n(t) - W_n^l(t) - W_n^o(t)]^+ + A_n(t), \quad (9)$$

where $[G_n(t) - W_n^l(t) - W_n^o(t)]^+ = \max\{G_n(t) - W_n^l(t) - W_n^o(t), 0\}$, $W_n^l(t) \leq G_n(t)$, and $W_n^o(t) \leq G_n(t) - W_n^l(t)$.

D. Energy Consumption Model

In the aerial-based MEC system, we investigate the energy consumption of GDs. The energy consumption can be divided into two components, including the energy consumed for local computation and offloading. The energy consumed by local computation hinges on the integrated chip architecture of the GDs [7]. Then, the energy consumed by local computation of GD n is calculated as

$$E_n^l(t) = \kappa f_n^l(t)^3 \tau, \quad (10)$$

where κ means the effective switched capacitance [21].

Similarly, the task offloading from GDs to servers consumes the corresponding energy, which is

$$E_n^o(t) = p_n(t)\tau. \quad (11)$$

Therefore, the total energy consumed by the GD n during slot t is

$$E_n(t) = E_n^l(t) + E_n^o(t). \quad (12)$$

Hence, the total energy consumed by GDs within slot t is represented as

$$E(t) = \sum_{n=1}^N E_n(t). \quad (13)$$

E. Problem Formulation

The optimization objective is to minimize the overall energy consumed by local and offloading for GDs. The decision set is represented as $\mathcal{Q}(t) = \{\mathbf{f}^l(t), \mathbf{v}(t), \mathbf{p}(t)\}$. According to the description above, this problem is formulated as

$$\mathcal{P}_1 : \min_{\mathcal{Q}(t)} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\{E(t)\}, \quad (14)$$

$$\begin{aligned} \text{s.t. } & C1 : 0 \leq f_n^l(t) \leq f_n^{\max}, \forall n \in \mathcal{N}, \\ & C2 : v_n^s(t) \in \{0, 1\}, \forall n \in \mathcal{N}, s \in \mathcal{S}, \\ & C3 : \sum_{s=0}^S v_n^s(t) \leq 1, \forall n \in \mathcal{N}, s \in \mathcal{S}, \\ & C4 : 0 \leq p_n(t) \leq p_n^{\max}, \forall n \in \mathcal{N}, \\ & C5 : W_n^l(t) \leq G_n(t), \forall n \in \mathcal{N}, \\ & C6 : W_n^o(t) \leq G_n(t) - W_n^l(t), \forall n \in \mathcal{N}, \end{aligned}$$

where $C1$ is local CPU cycle frequency constraint, $C2$ and $C3$ denote server selection decision constraint, and $C4$ is the power constraint. $C5$ and $C6$ are local computation constraint and offload computation constraint, respectively.

Problem $\mathcal{P}_1$ is a stochastic optimization problem, and it is an NP-hard problem involving mixed integer and continuous variable. The statistical data, such as the arrival process of task and task size of GDs, and wireless channel quality, are difficult to predict. Because of the unpredictability of the future information of the system, we will next use stochastic optimization techniques to deal with the problem.

III. DISTRIBUTED ONLINE TASK OFFLOADING AND RESOURCE ALLOCATION ALGORITHM DESIGN

This section innovates stochastic optimization techniques to transform $\mathcal{P}_1$ into a deterministic optimization problem. Then, a distributed online algorithm is designed to minimize the energy consumed by GDs. The algorithm makes decisions without relying on future information.

A. Problem Transformation

A queue backlog vector $\Omega(t) = \{G_1(t), G_2(t), \dots, G_N(t)\}$ describes the congestion level and stability level of the system, and $\Psi(\Omega(t)) = \frac{1}{2} \sum_{n=1}^N G_n(t)^2$.

It is clear that the function $\Psi(\Omega(t))$ is non-negative. Its larger value indicates that the GD is backlogged with more tasks, which will lead to system instability and affect the performance of the GD. To stabilize the queue level, the drift function is further

depicted as

$$\Delta(\Omega(t)) = \mathbb{E}\{\Psi(\Omega(t+1)) - \Psi(\Omega(t))|\Omega(t)\}. \quad (15)$$

By combining queue stability with energy consumption cost, the drift-plus-penalty function can be obtained as

$$\Delta_V(\Omega(t)) = \Delta(\Omega(t)) + V\mathbb{E}\{E(t)|\Omega(t)\}, \quad (16)$$

where the penalty weight V is a trade-off parameter.

Theorem 1: Regardless of the queue backlog and feasible decisions change, the upper bound of (16) satisfies

$$\begin{aligned} \Delta(\Omega(t)) + V\mathbb{E}\{E(t)|\Omega(t)\} &\leq O1 + V\mathbb{E}\{E(t)|\Omega(t)\} \\ &+ \mathbb{E}\left\{\sum_{n=1}^N G_n(t)[A_n(t) - W_n^l(t) - W_n^o(t)]|\Omega(t)\right\}, \end{aligned} \quad (17)$$

where $O1 = \sum_{n=1}^N \{\frac{1}{2}(A_n^{\max}(t))^2 + \frac{1}{2}[\frac{f_n^{\max}\tau}{\varrho_n} + R_n^{\max}(t)\tau]^2\}$ is a constant.

Proof: Squaring both sides of (9) simultaneously can yield

$$\begin{aligned} G_n(t+1)^2 &\leq G_n(t)^2 + [W_n^l(t) + W_n^o(t)]^2 + A_n(t)^2 \\ &+ 2G_n(t)[A_n(t) - W_n^l(t) - W_n^o(t)]. \end{aligned} \quad (18)$$

Integrating inequalities (15), (16) and (18) can yield

$$\begin{aligned} \Delta(\Omega(t)) + V\mathbb{E}\{E(t)|\Omega(t)\} &\leq O1 \\ &+ V\mathbb{E}\left\{\sum_{n=1}^N [E_n^l(t) + E_n^o(t)]|\Omega(t)\right\} \\ &+ \mathbb{E}\left\{\sum_{n=1}^N G_n(t)[A_n(t) - W_n^l(t) - W_n^o(t)]|\Omega(t)\right\}, \end{aligned} \quad (19)$$

where $O1 = \sum_{n=1}^N \{\frac{1}{2}(A_n^{\max}(t))^2 + \frac{1}{2}[\frac{f_n^{\max}\tau}{\varrho_n} + R_n^{\max}(t)\tau]^2\}$ is a constant. ■

With the above transformation, the problem $\mathcal{P}_1$ is transformed into the more tractable problem $\mathcal{P}_2$, which is denoted as follows

$$\begin{aligned} \mathcal{P}_2 : \min_{\mathbf{Q}(t)} &\left\{ V \sum_{n=1}^N [\kappa f_n^l(t)^3 \tau + p_n(t)\tau] \right. \\ &\left. - \sum_{n=1}^N G_n(t) \left[\frac{f_n^l(t)\tau}{\varrho_n} + \sum_{s=0}^S R_n^s(t)\tau \right] \right\}, \end{aligned} \quad (20)$$

$$\text{s.t. } C1 : 0 \leq f_n^l(t) \leq f_n^{\max}(t), \forall n \in \mathcal{N},$$

$$C2 : v_n^s(t) \in \{0, 1\}, \forall n \in \mathcal{N}, s \in \mathcal{S},$$

$$C3 : \sum_{s=0}^S v_n^s(t) \leq 1, \forall n \in \mathcal{N}, s \in \mathcal{S},$$

$$C4 : 0 \leq p_n(t) \leq p_n^{\max}, \forall n \in \mathcal{N},$$

$$C5 : W_n^l(t) \leq G_n(t), \forall n \in \mathcal{N},$$

$$C6 : W_n^o(t) \leq G_n(t) - W_n^l(t), \forall n \in \mathcal{N}.$$

B. Distributed Online Task Offloading and Resource Allocation Algorithm

This section investigates how to tackle problem $\mathcal{P}_2$ and propose the corresponding algorithm. First, we separate $\mathcal{P}_2$ into two subproblems, which are local computation resource allocation subproblem $\mathcal{P}_{2.1}$ and offloading resource allocation subproblem $\mathcal{P}_{2.2}$. Then, different methods are used to solve the problem according to the properties of the subproblems. Finally, the Distributed Online Task Offloading and Resource Allocation (DOTORA) algorithm is proposed.

1) *Local Computation Resource Allocation:* The local computation resource allocation subproblem is denoted as

$$\mathcal{P}_{2.1} : \min_{f^l(t)} \sum_{n=1}^N \left[V \kappa f_n^l(t)^3 \tau - G_n(t) \frac{f_n^l(t)\tau}{\varrho_n} \right], \quad (21)$$

$$\text{s.t. } C1 : 0 \leq f_n^l(t) \leq f_n^{\max}, \forall n \in \mathcal{N},$$

$$C2 : W_n^l(t) \leq G_n(t), \forall n \in \mathcal{N}.$$

This problem is decoupled for each GD. That is easy to see that $\mathcal{P}_{2.1}$ is a convex optimization problem regarding $f^l(t)$, and its solution can be obtained at the poles or at the boundary. We can obtain its pole value by assigning its first-order derivative function to zero. Then, the solution can be obtained as follows

$$f_n^{l,*}(t) = \begin{cases} \sqrt{\frac{G_n(t)}{3V\kappa\varrho_n}}, & 0 \leq \sqrt{\frac{G_n(t)}{3V\kappa\varrho_n}} \leq f_n^l(t) \\ f_n^l(t), & \text{otherwise} \end{cases}, \quad (22)$$

where $f_n^l(t) = \min\{\frac{G_n(t)\varrho_n}{\tau}, f_n^{\max}\}$.

After obtaining the local CPU frequency of the GDs and bringing it into (7), the local computation $W_n^l(t)$ of each GD can be calculated.

2) *Offloading Resource Allocation:* The server selection decision and transmission power allocation decision for GDs can be obtained by solving $\mathcal{P}_{2.2}$.

$$\mathcal{P}_{2.2} : \min_{\mathbf{v}(t), \mathbf{p}(t)} \sum_{n=1}^N \left[V p_n(t)\tau - G_n(t) \sum_{s=0}^S R_n^s(t)\tau \right], \quad (23)$$

$$\text{s.t. } C1 : v_n^s(t) \in \{0, 1\}, \forall n \in \mathcal{N}, s \in \mathcal{S},$$

$$C2 : \sum_{s=0}^S v_n^s(t) \leq 1, \forall n \in \mathcal{N}, s \in \mathcal{S},$$

$$C3 : 0 \leq p_n(t) \leq p_n^{\max}, \forall n \in \mathcal{N},$$

$$C4 : W_n^o(t) \leq G_n(t) - W_n^l(t), \forall n \in \mathcal{N}.$$

Solving problem $\mathcal{P}_{2.2}$ is very difficult because the decisions $\mathbf{v}(t)$ and $\mathbf{p}(t)$ are coupled. Thus, we employ game theory to reformulate the problem as a game-theoretical multi-server selection game. Then, we use a game approach to obtain the server chosen by each GD. Here, we look for a server selection decision set that minimizes the total offloading cost. After all GDs have been allocated servers, the total offloading cost is minimized by continuously adjusting the transmission power of the GDs.

a) *Game-theoretical Multi-server Selection:* To solve the server selection problem of GDs, in this part, we model the server selection problem as a non-cooperative game model using game theory [22], [23], namely GMS game. At this stage, each GD n is assigned a default transmission power $p_n^{def}(t)$. Specifically, when $t = 0$, $p_n^{def}(t) = 0$; otherwise, $p_n^{def}(t) = p_n(t - 1)$. For the GD n , the decisions of other GDs are denoted as $\mathbf{v}_{-n}(t)$. Then the total cost of offloading is

$$\mathcal{P}_{2.2.1} : \min_{\mathbf{v}(t)} \sum_{n=1}^N [C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}(t))], \quad (24)$$

$$\text{s.t. } C1 : v_n^s(t) \in \{0, 1\}, \forall n \in \mathcal{N}, s \in \mathcal{S},$$

$$C2 : \sum_{s=0}^S v_n^s(t) \leq 1, \forall n \in \mathcal{N}, s \in \mathcal{S},$$

$$C3 : W_n^o(t) \leq G_n(t) - W_n^l(t), \forall n \in \mathcal{N},$$

where

$$\begin{aligned} C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}(t)) \\ = V p_n^{def}(t) \tau - G_n(t) \sum_{s=0}^S R_n^s(t) \tau = V p_n^{def}(t) \tau \\ - G_n(t) \sum_{s=0}^S B_s \log_2 \left(1 + \frac{v_n^s(t) p_n^{def}(t) g_n^s(t)}{\sum_{m \neq n} v_m^s(t) p_m^{def}(t) g_m^s(t) + \sigma^2} \right) \tau. \end{aligned} \quad (25)$$

Within each time slot, GD n will make an appropriate server selection decision. Then this server selection problem is modeled as a game $F = (N, \{\mathbf{v}_n(t)\}_{n \in \mathcal{N}}, \{C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}(t))\}_{n \in \mathcal{N}})$, where N denotes the number of GDs in the system, $\mathbf{v}_n(t)$ is the server selection decision of GD n , and $C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}(t))$ denotes the offloading cost for GD n to make the decision $\mathbf{v}_{-n}(t)$ when the decisions of other GDs are given.

In the game F , when multiple GDs select to offload tasks to the same HAP or UAV, there is noncooperative competition among these several GDs. To describe this noncooperative competition, we define the Nash Equilibrium (NE) solution.

Definition 1: A server selection decision $\mathbf{v}^*(t) = \{\mathbf{v}_1^*(t), \mathbf{v}_2^*(t), \dots, \mathbf{v}_n^*(t)\}$ in the GMS game is a NE when no GD can further reduce its offloading cost by independently altering its decision, i.e.

$$C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}^*(t)) > C_n(\mathbf{v}_n^*(t), \mathbf{v}_{-n}^*(t)), \forall n \in \mathcal{N}. \quad (26)$$

In the NE, each GD will no longer change its strategy to reduce offloading cost, i.e., all GDs find their own optimal decisions. Next, we will investigate whether there exists NE in the game F . If there exists NE, then the decisions of all GDs will self-organize into NE in finite iterations based on the best-want principle.

In the following, we will introduce a potential game to demonstrate the presence of NE in F . The potential game possesses finite improvement property, guaranteeing the existence of at least one NE within it. Therefore, we can prove the presence of

NE in the GMS game by showing that the GMS game aligns with those typically associated with a potential game.

Definition 2: F is a potential game when there exists a function that satisfies

$$\begin{aligned} C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}(t)) - C_n(\mathbf{v}'_n(t), \mathbf{v}_{-n}(t)) > 0 \\ \Rightarrow \Phi(\mathbf{v}_n(t), \mathbf{v}_{-n}(t)) - \Phi(\mathbf{v}'_n(t), \mathbf{v}_{-n}(t)) > 0. \end{aligned} \quad (27)$$

Theorem 2: F is a potential game with a corresponding potential function represented as

$$\Phi(\mathbf{v}_n(t), \mathbf{v}_{-n}(t)) = \log_2 \left(\frac{\sum_{m \neq n} v_m^s(t) p_m^{def}(t) g_m^s(t) + \sigma^2}{\sum_{n \in \mathcal{N}} v_n^s(t) p_n^{def}(t) g_n^s(t) + \sigma^2} \right). \quad (28)$$

Proof: Based on the above discussion, we can conclude that

$$\begin{aligned} C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}(t)) &= V p_n^{def}(t) \tau \\ &- G_n(t) \sum_{s=0}^S B_s \log_2 \left(1 + \frac{v_n^s(t) p_n^{def}(t) g_n^s(t)}{\sum_{m \neq n} v_m^s(t) p_m^{def}(t) g_m^s(t) + \sigma^2} \right) \tau. \end{aligned} \quad (29)$$

The difference between server selection decisions $\mathbf{v}_n(t)$ and $\mathbf{v}'_n(t)$ is calculated as

$$\begin{aligned} C_n(\mathbf{v}_n(t), \mathbf{v}_{-n}(t)) - C_n(\mathbf{v}'_n(t), \mathbf{v}_{-n}(t)) \\ = -G_n(t) \sum_{s=0}^S B_s \log_2 \left(1 + \frac{v_n^s(t) p_n^{def}(t) g_n^s(t)}{\sum_{m \neq n} v_m^s(t) p_m^{def}(t) g_m^s(t) + \sigma^2} \right) \tau \\ + G_n(t) \sum_{s=0}^S B_s \log_2 \left(1 + \frac{v_n^{s'}(t) p_n^{def}(t) g_n^{s'}(t)}{\sum_{i \neq n} v_i^{s'}(t) p_i^{def}(t) g_i^{s'}(t) + \sigma^2} \right) \tau \\ = G_n(t) \sum_{s=0}^S B_s \tau \left[\log_2 \left(1 + \frac{v_n^{s'}(t) p_n^{def}(t) g_n^{s'}(t)}{\sum_{i \neq n} v_i^{s'}(t) p_i^{def}(t) g_i^{s'}(t) + \sigma^2} \right) \right. \\ \left. - \log_2 \left(1 + \frac{v_n^s(t) p_n^{def}(t) g_n^s(t)}{\sum_{m \neq n} v_m^s(t) p_m^{def}(t) g_m^s(t) + \sigma^2} \right) \right] \\ > 0. \end{aligned} \quad (30)$$

When the server selection decision of GD n changes from $\mathbf{v}_n(t)$ to $\mathbf{v}'_n(t)$, the potential function changes as follows

$$\begin{aligned} \Phi(\mathbf{v}_n(t), \mathbf{v}_{-n}(t)) - \Phi(\mathbf{v}'_n(t), \mathbf{v}_{-n}(t)) \\ = \log_2 \left(\frac{\sum_{m \neq n} v_m^s(t) p_m^{def}(t) g_m^s(t) + \sigma^2}{\sum_{n \in \mathcal{N}} v_n^s(t) p_n^{def}(t) g_n^s(t) + \sigma^2} \right) \\ - \log_2 \left(\frac{\sum_{m \neq n} v_m^{s'}(t) p_m^{def}(t) g_m^{s'}(t) + \sigma^2}{\sum_{n \in \mathcal{N}} v_n^{s'}(t) p_n^{def}(t) g_n^{s'}(t) + \sigma^2} \right) \\ > 0. \end{aligned} \quad (31)$$

Through the above derivation, we can see that the trend of cost function and potential function is consistent. Therefore, Theorem 2 is proven. ■

Algorithm 1: Distributed Game-Theoretical Multi-Server Selection (DGMS) Algorithm.

Input: $\mathcal{S}, \mathcal{N}$, default transmission power $p(t)$
Output: server selection decision $v(t)$

- 1: **repeat**
- 2: calculate the current total offloading cost $\sum_{n=1}^N C(v_n(t), v_{-n}(t))$
- 3: **for all** $n \in \mathcal{N}$ **do**
- 4: **for all** $s \in \mathcal{S}$ **do**
- 5: calculate the total cost $\sum_{n=1}^N C(v_n'(t), v_{-n}(t))$ when user n selects server s
- 6: **end for**
- 7: find decision $v_n'(t)$ with the minimum total offloading cost
- 8: **if** $\sum_{n=1}^N C(v_n'(t), v_{-n}(t)) < \sum_{n=1}^N C(v_n(t), v_{-n}(t))$ **then**
- 9: temporarily update the decision for GD n to $v_n'(t)$
- 10: **end if**
- 11: **end for**
- 12: randomly selects one GD among all GDs whose decisions are to be updated to update its decision
- 13: **until** all GDs no longer update decisions

To solve the server selection problem for GDs, the DGMS algorithm is proposed, the details of which can be found in Algorithm 1.

b) Transmission Power Allocation: After the GD determines which UAV or HAP to offload the task, we can adjust the transmission power according to their respective needs to reduce the offloading cost. The transmission power allocation problem is modeled as

$$\mathcal{P}_{2.2.2} : \min_{p(t)} \left\{ V p_n(t) \tau - G_n(t) B_s \log_2 \left(1 + \frac{p_n(t) g_n^s(t)}{\sum_{m \neq n} p_m(t) g_m^s(t) + \sigma^2} \right) \tau \right\}, \quad (32)$$

$$\begin{aligned} \text{s.t. } C1 : 0 \leq p_n(t) \leq p_n^{\max}, \forall n \in \mathcal{N}, \\ C2 : W_n^o(t) \leq G_n(t) - W_n^l(t), \forall n \in \mathcal{N}. \end{aligned}$$

To obtain the solution of the transmission power, first, we need to acquire the power expression for each GD. Then, the default transmission power is set for all GDs. Finally, the satisfactory transmission powers of all GDs are obtained by finite iterations.

Problem $\mathcal{P}_{2.2.2}$ is not easy to solve. Thus, let $K_n(t) = G_n(t) B_s$, $r_n(t) = \frac{g_n^s(t)}{\sum_{m \neq n} p_m(t) g_m^s(t) + \sigma^2}$, we can get

$$H(p_n(t)) = -K_n(t) \log_2(1 + p_n(t) r_n(t)) \tau + V p_n(t) \tau, \quad (33)$$

$$\begin{aligned} \text{s.t. } C1 : 0 \leq p_n(t) \leq p_n^{\max}, \forall n \in \mathcal{N}, \\ C2 : W_n^o(t) \leq G_n(t) - W_n^l(t), \forall n \in \mathcal{N}. \end{aligned}$$

Algorithm 2: Transmission Power Allocation (TPA) Algorithm.

Input: $\mathcal{S}, \mathcal{N}$, default transmission power $p(t)$, server selection decision $v(t)$ found in Algorithm 1
Output: transmission power allocation decision $p'(t)$

- 1: **repeat**
- 2: calculate the current cost $\sum_{n=1}^N H(p(t))$
- 3: **for all** $n \in \mathcal{N}$ **do**
- 4: calculate $K_n(t)$ and $r_n(t)$
- 5: **if** $K_n(t) = 0$ **then**
- 6: set $p'_n(t) = 0$
- 7: **end if**
- 8: **if** $K_n(t) > 0$ and $V \geq \frac{K_n(t) r_n(t)}{\ln 2}$ **then**
- 9: set $p'_n(t) = 0$
- 10: **else**
- 11: set $p'_n(t) = \min\{p_n^{\max}, \frac{K_n(t)}{V \ln 2} - \frac{1}{r_n(t)}\}$
- 12: **end if**
- 13: **end for**
- 14: calculate the new total offloading cost $\sum_{n=1}^N H(p'(t))$ after updating the transmission power of all GDs
- 15: **until** $\sum_{n=1}^N H(p'(t)) \leq \sum_{n=1}^N H(p(t))$ and $\|p(t) - p(t-1)\| < \epsilon$

For the convenience of description, we obtain $H(p_n(t))$ through equivalent transformation. The solution of problem $\mathcal{P}_{2.2.2}$ is the same as the solution corresponding to the minimum value of function $H(p_n(t))$. Therefore, we can obtain the optimal solution of problem $\mathcal{P}_{2.2.2}$ by solving the solution corresponding to the minimum value of function $H(p_n(t))$.

The first and second derivatives of $H(p_n(t))$ are given by

$$\frac{dH(p_n(t))}{dp_n(t)} = V\tau - \frac{K_n(t) r_n(t) \tau}{\ln 2 [1 + p_n(t) r_n(t)]}, \quad (34)$$

$$\frac{d^2 H(p_n(t))}{d^2 p_n(t)} = \frac{K_n(t) r_n(t)^2 \tau}{\ln 2 [1 + p_n(t) r_n(t)]^2}. \quad (35)$$

Two cases are considered, as described below.

When $K_n(t) > 0$, the second derivative of $H(p_n(t))$ is greater than 0, $H(p_n(t))$ is a convex function. We can find that the zero point of the first derivative of $H(p_n(t))$ is $p_n(t) = \frac{K_n(t)}{V \ln 2} - \frac{1}{r_n(t)}$. Therefore, the solution as

$$p_n(t) = \begin{cases} 0, & V \geq \frac{K_n(t) r_n(t)}{\ln 2} \\ \min\{p_n^{\max}, \frac{K_n(t)}{V \ln 2} - \frac{1}{r_n(t)}\}, & \text{otherwise} \end{cases}. \quad (36)$$

When $K_n(t) = 0$, the first derivative of $H(p_n(t))$ is greater than 0, indicating that $H(p_n(t))$ is a monotonically increasing function. Thus, $p_n(t) = 0$.

To address the TPA problem of GDs, we propose the TPA algorithm. Algorithm 2 gives the details of TPA.

The relationship among the series of optimization subproblems is summarized in Fig. 2.

Next, we propose the DOTORA algorithm. In each time slot, the algorithm determines the CPU cycle frequency of the GD based on the amount of arriving tasks, and makes the server selection decision and transmission power allocation decision of

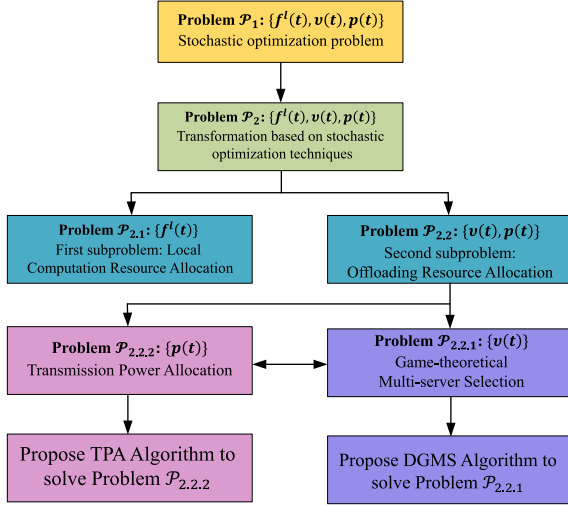


Fig. 2. Relationship among optimization problems.

Algorithm 3: Distributed Online Task Offloading and Resource Allocation (DOTORA) Algorithm.

Input: $\mathcal{S}, \mathcal{N}, \kappa$ **Output:** $f_n^l(t), v_n^s(t), p_n(t), \forall n \in \mathcal{N}, s \in \mathcal{S}$

```

1: for all  $t \in \mathcal{T}$  do
2:   obtain  $A_n(t)$ 
3:   for all  $n \in \mathcal{N}$  do
4:     calculate  $f_n^l(t)$  by (22)
5:   end for
6:   determine  $v(t)$  by Algorithm 1
7:   determine  $p(t)$  by Algorithm 2
8:   update  $G_n(t+1), \forall n \in \mathcal{N}$ 
9: end for

```

the GD by calling the DGMS algorithm and the TPA algorithm respectively. This algorithm can minimize the drift plus penalty of each time slot, effectively solving problem $\mathcal{P}_1$.

In each time slot, the DGMS algorithm takes the server selection decision set and transmission power allocation decision set obtained in the previous time slot as algorithm input. Then, based on the current system status, the DGMS algorithm calculates a new server selection decision set and takes it as output. The TPA algorithm takes the transmission power allocation decision set obtained in the previous time slot and the new server selection decision set obtained by the DGMS algorithm in the current time slot as algorithm inputs. Afterwards, based on the current system status, the TPA algorithm calculates a new transmission power allocation decision set and takes it as output. The DOTORA algorithm incorporates the DGMS and TPA algorithms as sub-algorithms, and it utilizes the two algorithms to obtain the server selection decision set and transmission power allocation decision set.

Then, we give the complexity analysis of the proposed algorithm. The complexity of solving problem $\mathcal{P}_{2.1}$ is $\mathcal{O}(N)$. For the Algorithm 1, in each round, we find the best server for each GD by looking at all possible options (lines 3–11 of Algorithm 1) to find the lowest cost server selection decision.

The complexity of the part is $\mathcal{O}(N^2 S^2)$. In addition, in Theorem 2 in the revised paper, we prove that the GMS game is a potential game. Therefore, the game has finite improvement property (FIP), that is, the NE solution can be obtained through a finite number of iterations. Let the number of iterations be represented by K_1 . The complexity of the Algorithm 1 can be expressed as $\mathcal{O}(K_1 N^2 S^2)$. For the Algorithm 2, an iterative method is used to solve the transmission power of each GD, and the maximum number of iterations is K_2 . In addition, the complexity of solving the minimum cost for each iteration is $\mathcal{O}(NS)$. Then the complexity of Algorithm 2 can be expressed as $\mathcal{O}(K_2 NS)$. Thus, the complexity of the proposed DOTORA algorithm is $\mathcal{O}(N + K_1 N^2 S^2 + K_2 NS)$.

We also give an analysis of communication overhead. In each time slot, GD makes local computing decisions, and the process involves no communication overhead. Then, when GD performs task offloading, it needs to obtain server selection decisions and transmission power allocation decisions. This process involves communication overhead:

(1) The server selection decision is obtained by calling the Algorithm 1. In the first step, in each iteration, GD sends its task status information to each server (including the amount of newly arrived tasks, queue backlog, server selection decision for this iteration, etc.). In the second step, the server sends the processed information to GD, and GD can obtain the status information and decisions of other GDs. In the third step, GD sends a request update decision message to the server. In the fourth step, when the server collects all the information containing the GD's desired decision, the server selects a GD that can update its server selection decision (the winner in the Algorithm 1), and sends a message containing the updated decision to each GD. In the fifth step, in Theorem 2, we prove that the GMS game is a potential game. Therefore, the game has finite improvement property (FIP), that is, a NE solution can be obtained through a finite number of iterations. Let R_1 represent the number of iterations, then the communication overhead of this part is $\mathcal{O}((3N + NS)R_1) = \mathcal{O}(NSR_1)$.

(2) The transmission power allocation decision is obtained by calling the Algorithm 2. In the first step, GD sends information (including the amount of newly arrived tasks, queue backlog, power allocation decision for this iteration, etc.) to the server it selected. In the second step, the server sends the information of the GD it serves to other GDs. In the third step, GD makes decisions based on this information and sends updated decision information to the server. Assuming that R_2 iterations are required to obtain the power allocation decision, the communication overhead of this part is $\mathcal{O}(3NR_2)$.

Thus, the total communication overhead is $\mathcal{O}(NSR_1 + 3NR_2)$.

IV. PERFORMANCE ANALYSIS OF DOTORA ALGORITHM

This section analyzes the performance of the DOTORA mathematically. First, the time-average queue backlog is denoted as

$$\bar{G} = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{n=1}^N G_n(t). \quad (37)$$

Lemma 1: Within any time slot, for a random arrival rate δ , there is a corresponding offloading decision ζ^* such that

$$\begin{aligned}\mathbb{E}\{E^{\zeta^*}(t)\} &= E^*(\delta), \\ \mathbb{E}\{A_n^{\zeta^*}(t)\} &\leq \mathbb{E}\{W_n^{l,\zeta^*}(t) + W_n^{o,\zeta^*}(t)\},\end{aligned}\quad (38)$$

where $E^*(\delta)$ expresses minimum total energy consumption.

Proof: Caratheodory's theorem can be adopted to demonstrate Lemma 1. We omit the details of the proof. ■

It is important to note that the task arrival rate's upper bound implies that the total energy consumption of GDs is upper bounded, and its upper and lower bounds are defined as $\hat{E}$ and $\check{E}$, respectively. Then, we give Theorem 3.

Theorem 3: For arbitrary V and task arrival rate $\delta + \varepsilon$, the average energy consumption and average queue backlog satisfy

$$E^{DOTORA} \leq E^* + \frac{O1 + O2}{V}, \quad (39)$$

$$\bar{G} \leq \frac{O1 + O2 + V(\hat{E} - \check{E})}{\varepsilon}, \quad (40)$$

where $O1 = \sum_{n=1}^N \{\frac{1}{2}(A_n^{\max}(t))^2 + \frac{1}{2}[\frac{f_n^{\max}\tau}{\theta_n} + R_n^{\max}(t)\tau]^2\}$ is a constant, $O2$ is the gap between the optimal result and the result with our DOTORA algorithm in subproblem $\mathcal{P}_{2.2}$, i.e., $\mathcal{P}_{2.2}$ is solved within an optimality gap $O2$.

Proof: By utilizing Lemma 1, for offloading decision ζ and arrival rate $\delta + \varepsilon$, we get

$$\begin{aligned}\mathbb{E}\{E^{\zeta}(t)\} &= E^*(\delta + \varepsilon), \\ \mathbb{E}\{A_n^{\zeta}(t)\} + \varepsilon &\leq \mathbb{E}\{W_n^{l,\zeta}(t) + W_n^{o,\zeta}(t)\}.\end{aligned}\quad (41)$$

Because the optimization objective of DOTORA is to minimize (19)'s R.H.S, for offloading decision ζ , it holds

$$\begin{aligned}\Delta(\Omega(t)) + V\mathbb{E}\{E(t)\} &\leq O1 + O2 + V\mathbb{E}\{E(t)\} \\ &+ \mathbb{E}\left\{\sum_{n=1}^N G_n(t)[A_n^{\zeta}(t) - W_n^{l,\zeta}(t) - W_n^{o,\zeta}(t)]\right\}.\end{aligned}\quad (42)$$

Substituting (41) into (42), we obtain

$$\begin{aligned}\mathbb{E}\{\Psi(\Omega(t+1)) - \Psi(\Omega(t))\} + V\mathbb{E}\{E(t)\} \\ \leq O1 + O2 + VE^*(\delta + \varepsilon) - \varepsilon \sum_{n=1}^N \mathbb{E}\{G_n(t)\}.\end{aligned}\quad (43)$$

As $G_n(t)$ and ε are not negative, by adding (43) of all $t \in \mathcal{T}$, there is

$$V \sum_{t=0}^{T-1} \mathbb{E}\{E(t)\} \leq (O1 + O2)T + VTE^*(\delta + \varepsilon). \quad (44)$$

Divide (44) by VT . When $\varepsilon \rightarrow 0$, $T \rightarrow \infty$, (39) is proven. Using (43), it holds

$$\begin{aligned}\varepsilon \sum_{t=0}^{T-1} \sum_{n=1}^N \mathbb{E}\{G_n(t)\} \\ \leq (O1 + O2)T + VTE^*(\delta + \varepsilon) - V \sum_{t=0}^{T-1} \mathbb{E}\{E(t)\}.\end{aligned}\quad (45)$$

TABLE II
PARAMETER SETTINGS

Parameters	Value
Effective switched capacitance	10^{-27}
Background noise power	10^{-13} W
Bandwidth of UAV	2 MHz
Bandwidth of HAP	10 MHz
CPU cycles needed to process 1 bit data	1000 cycles/bit
Carrier frequency	0.1 GHz
Height of HAP	20 Km
Height of UAV	200 m
Link loss η_n^{LoS} and η_n^{NLoS}	0.1, 21
Environment parameter $\varkappa_1, \varkappa_2$	4.88, 0.43
Convergence criteria ϵ	10^{-4}

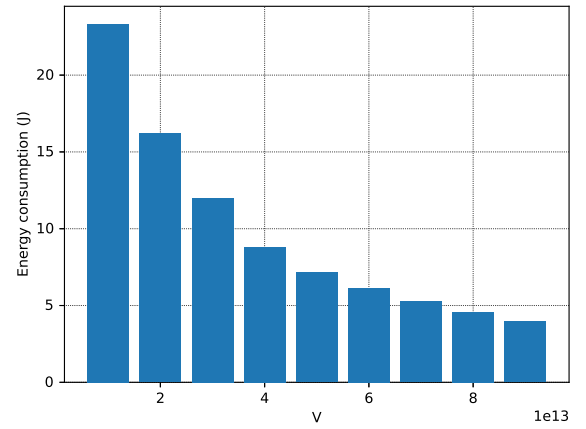


Fig. 3. Energy consumption v.s. V.

Since $E(t)$ is not negative, we have

$$\varepsilon \sum_{t=0}^{T-1} \sum_{n=1}^N \mathbb{E}\{G_n(t)\} \leq (O1 + O2)T + VT(\hat{E} - \check{E}). \quad (46)$$

Divide (46) by εT , when $T \rightarrow \infty$, we get (40). ■

V. EXPERIMENT EVALUATION

A. Experiment Settings

This section conducts experiments to evaluate the performance of DOTORA. Considering one HAP, four UAVs, and multiple GDs deployed in a 1 km² remote area. In this area, the GDs are randomly distributed. The UAVs are uniformly deployed in the area in a hovering manner. In addition, for each GD, the amount of arrival task is uniformly distributed within $[0, 2] \times 10^6$ bits, the maximum available CPU frequency and maximum transmission power are 1 GHz and 0.2 W, respectively [24], [25]. Table II provides a summary of the main parameter settings.

B. Parameter Analysis

1) *Impact of Parameter V:* Figs. 3 and 4 show the effect of the control parameter V on energy consumption and queue backlog.

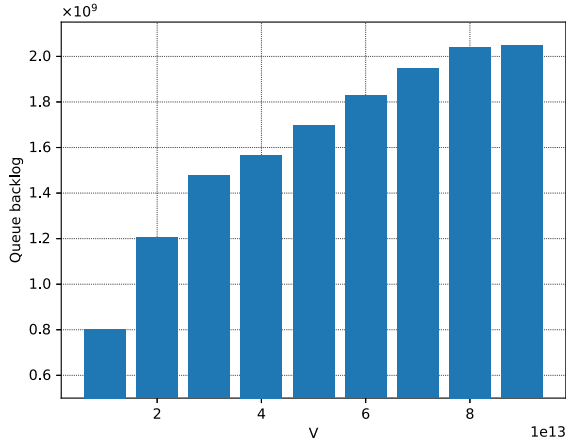


Fig. 4. Queue backlog v.s. V.

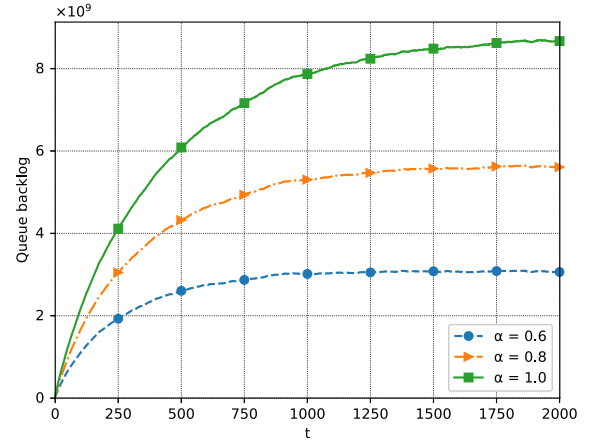


Fig. 6. Queue backlog v.s. task arrival rate.

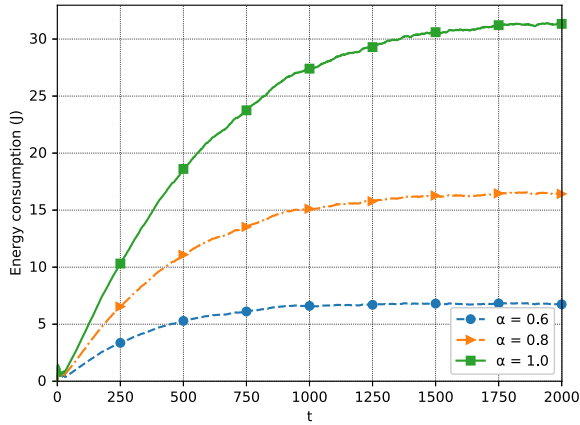


Fig. 5. Energy consumption v.s. task arrival rate.

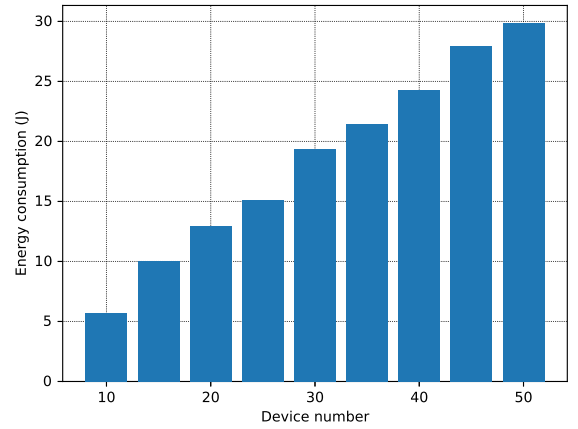


Fig. 7. Energy consumption v.s. number of GDs.

Fig. 3 shows that the energy consumption becomes smaller as V increases. This is owing to that the larger the control parameter V is, the higher the priority on energy consumption, and the system is more inclined to optimize energy consumption, coinciding with (39). Fig. 4 presents the change in queue backlog when V changes. We can see that as V grows, the average queue backlog becomes larger. This is owing to the limited computation and transmission capabilities of GDs, and the trend coincides with (40). In conclusion, the DOTORA algorithm reduces energy consumption of GD and ensures queue stability, achieving a trade-off between the two.

2) *Impact of Task Arrival Rate:* Figs. 5 and 6 show the effect of task arrival rate on energy consumption and queue backlog. The task arrival rate in this experiment is set to $\alpha \cdot A_n(t)$ and $\alpha = 0.6, 0.8$, and 1.0 . Fig. 5 shows that the energy consumption increases as α increase. This is because the more new tasks that arrive, the more tasks will be processed, resulting in higher energy consumption. Fig. 6 shows that as α increase, the queue backlog also increases accordingly. Because the processing capacity of GDs remains the same, when more tasks arrive, some of them cannot be processed in time, thus leading to an increase in queue backlog. However, in the long run, both

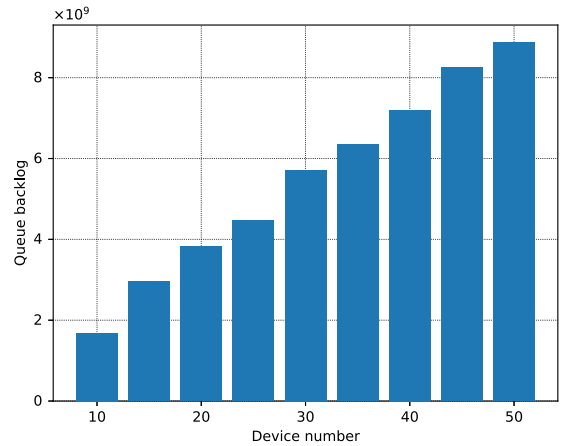


Fig. 8. Queue backlog v.s. number of GDs.

energy consumption and queue backlog tend to be stable, which validates the adaptivity of our DOTORA algorithm.

3) *Impact of Number of Device:* Figs. 7 and 8 show the change of energy consumption and queue backlog with different numbers of GDs. In our experiments, the device number ranges from 10 to 50. Fig. 7 illustrates that the more GDs access the

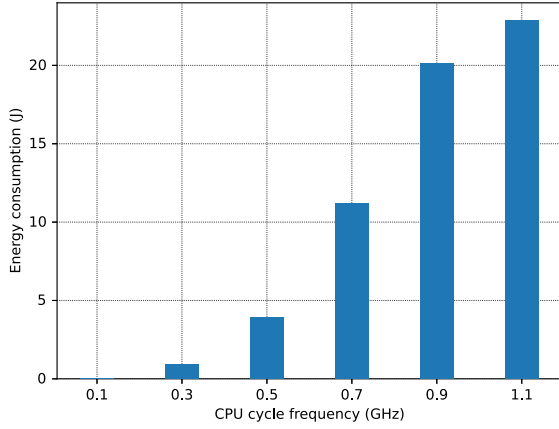


Fig. 9. Energy consumption v.s. CPU cycle frequency.

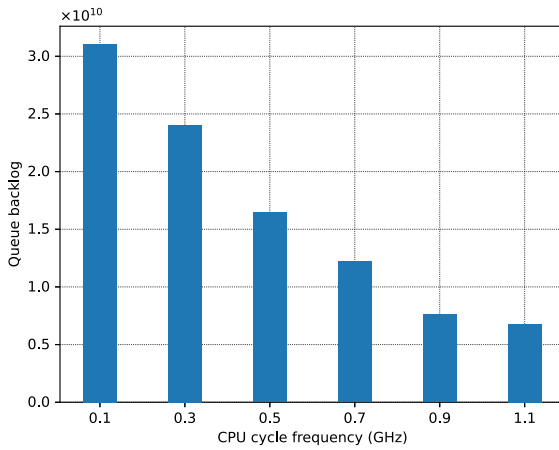


Fig. 10. Queue backlog v.s. CPU cycle frequency.

network, the more energy is consumed. This is because each GD consumes energy to process tasks, resulting in higher overall energy consumption as the number of GDs increases. Fig. 8 shows that the more GDs, the larger the queue backlog. This is owing to the fact that each GD has a queue backlog, which leads to an increase in the total queue backlog.

4) *Impact of the CPU Cycle Frequency of GDs:* Figs. 9 and 10 show the impact of changes in GD's CPU cycle frequency on energy consumption and queue backlog. In this experiment, the CPU cycle frequency of GD ranges from 0.1 GHz to 1.1 GHz. Fig. 9 shows that as the CPU cycle frequency of GD increases, the energy consumption also increases. This is obvious, because the greater the CPU cycle frequency of GD, the larger the task processing capability of GD. Furthermore, GD can handle more computing tasks, resulting in greater energy consumption. In addition, it can be seen from Fig. 10 that as the CPU cycle frequency of GD increases, the queue backlog gradually decreases. This is because GD has a larger CPU cycle frequency, which enables it to handle more tasks. Furthermore, there are fewer backlogged tasks, resulting in a smaller queue backlog. Taken together, both energy consumption and queue backlog have gradually stabilized, which verifies the effectiveness of our DOTORA algorithm.

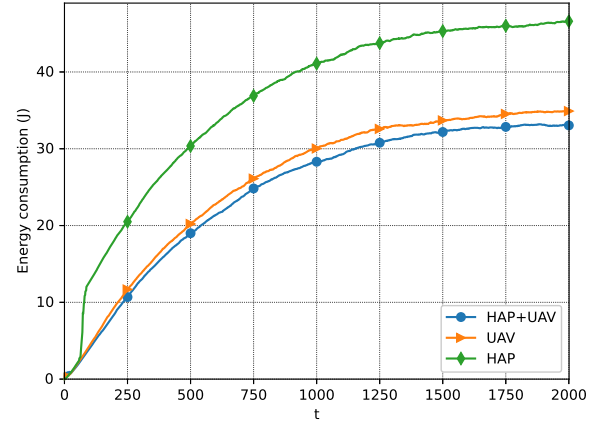


Fig. 11. Energy consumption v.s. different system frameworks.

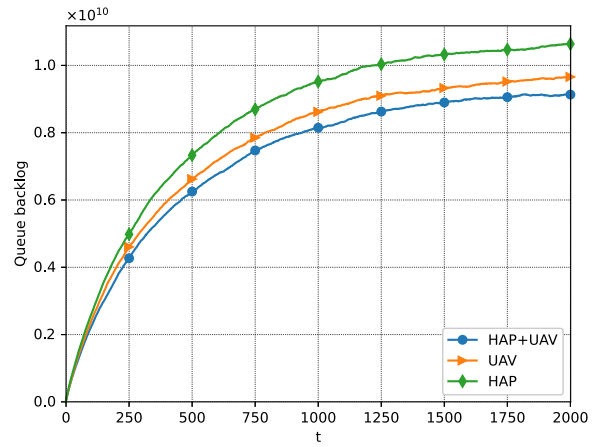


Fig. 12. Queue backlog v.s. different system frameworks.

C. Comparative Experiments

In this subsection, we evaluate different system frameworks. We compare the proposed aerial-based MEC system with a MEC system served by HAP or UAV only to verify that the aerial-based MEC system can effectively save the energy of GDs.

Figs. 11 and 12 depict the energy consumption and queue backlog under different system frameworks, respectively. UAV-Only means that the task can solely be transferred for processing to the UAV. HAP-Only represents that the task can only be offloaded to the HAP to complete the computation. HAP-UAV indicates that the HAP can cooperate with UAVs to accomplish the processing of the task. As observed from the figures, our proposed HAP-UAV framework effectively reduces the energy consumption of GDs. Because HAP and UAV work together to provide services to GDs, competition among GDs for communication and computing resources is reduced. Therefore, the energy consumption and task backlog of GDs are reduced, gaining a significantly improved user experience for GDs.

Then, the efficiency and effectiveness of the DOTORA algorithm are further verified by comparing the DOTORA algorithm with the following algorithms.

- *Full Local Computing (FLC):* The newly arrived tasks of each GD are processed by itself.

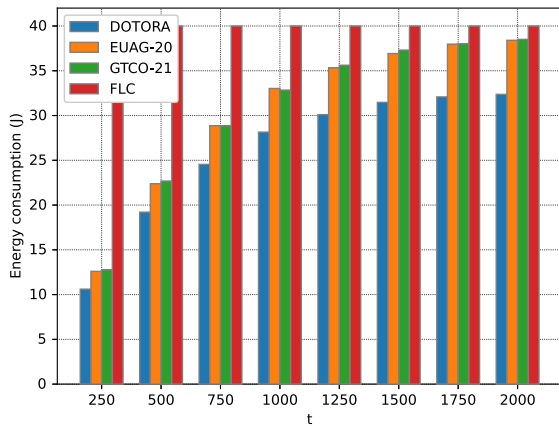


Fig. 13. Energy consumption v.s. different algorithms.

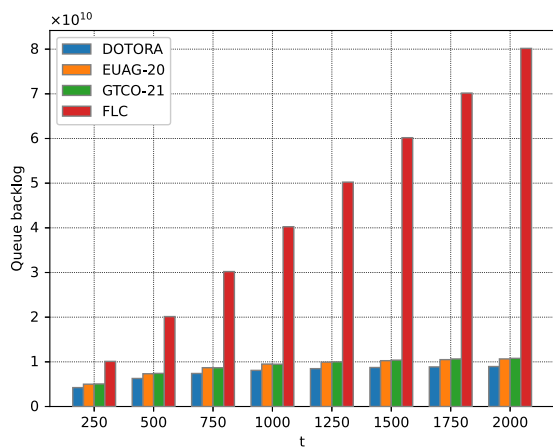


Fig. 14. Queue backlog v.s. different algorithms.

- *EUAG-20*: This algorithm is extended from [26] to our model, and we randomly select one GD each time to update its server selection decision.
- *GTCO-21*: Within each time slot, GDs determine a server selection strategy to minimize the offloading cost in each iteration based on [27].

Figs. 13 and 14 show the energy consumption and queue backlog under the four algorithms. As observed in Fig. 13, the DOTORA algorithm has the lowest energy consumption, the FLC algorithm has the highest energy consumption, and the GTCO-21 algorithm and EUAG-20 algorithm are almost the same. For the FLC algorithm, as time passes, the available computing power of GDs is fully utilized, and the amount of tasks that can be processed reaches its limit, the energy consumption of the FLC algorithm no longer changes. At the same time, we can see from Fig. 13 that compared with the EUAG-20 algorithm, GTCO-21 algorithm and FLC algorithm, the DOTORA algorithm has reduced energy consumption by 15.73%, 15.95% and 19.09%, respectively.

From Fig. 14, it can be seen that in terms of queue backlog, the DOTORA algorithm is the smallest, the FLC algorithm is the largest, and the GTCO-21 algorithm and EUAG-20 algorithm are almost the same. For the FLC algorithm, when the amount

of tasks that can be processed by GDs reaches its limit, newly arrived tasks will be cached in the queue for processing, causing the queue backlog to continue to increase. Because of the DOTORA algorithm's ability to dynamically adjust offloading strategies based on the current system state, its energy consumption and queue backlog are smaller than those of the GTCO-21 algorithm and EUAG-20 algorithm. In conclusion, the DOTORA algorithm can reduce energy consumption with the minimum queue backlog. Meanwhile, according to Fig. 14, we can see that compared with the EUAG-20 algorithm, GTCO-21 algorithm and FLC algorithm, the DOTORA algorithm reduces the queue backlog by 15.99%, 17.64% and 88.84%, respectively.

VI. RELATED WORK

UAV-assisted MEC networks have become one of the hot spots for research in recent years. There are many areas outside the coverage of MEC networks, such as wilderness, desert, etc. In these places, it is very difficult and expensive to establish a ground communication system. Meanwhile, the ground communication system may be damaged and unusable after sudden disasters. Therefore, the UAV-assisted MEC network becomes an alternative solution. Jeong et al. [28] investigated UAV-based cloud computing systems designed to enhance coverage or relay services via UAVs for users with limited or no infrastructure. Dai et al. [29] investigated data offloading from UAVs as relay nodes to address offloading rates and delays in offshore area communication. Seid et al. [30] considered the dynamics of channel strength and resource requests for UAVs, aiming to reduce system costs and ensure quality of service (QoS) requirements for GDs. These studies only focused on drone networks. However, the available energy and computing power of UAVs are limited, and they cannot provide long-term and sustainable services for GDs.

Compared with UAVs, HAPs have higher flying altitudes, and stronger payloads, and can better provide the required services for GDs in remote areas. Wang et al. [31] considered the situation where the size of computing tasks fluctuated over time in a HAP-supported network. The authors proposed an optimization problem and a joint learning solution based on a support vector machine, aiming to minimize the cost. Ke et al. [32] introduced an edge computing paradigm based on a high-altitude platform network, which used massive aerial multiple-input multiple-output (MIMO) technology to achieve large-scale access and computing services, and solved the energy and time-consuming problem of large-scale connection. Ren et al. [33] proposed a HAP-based edge intelligent computing framework that used caching and reinforcement learning techniques to optimize computation offloading and resource allocation to reduce transmission delay.

Nevertheless, compared to UAVs, HAPs are more expensive to deploy. When there are too many GDs, it will cause network congestion, and the integration of UAVs can reduce the burden on the network. Therefore, the research on integrated aerial computing combined with HAP and UAV has also received widespread attention. Kang et al. [16] proposed a system based on UAVs and HAPs, which achieved efficient processing and

QoS guarantee of computing tasks by jointly optimizing resource allocation and task offloading. Jia et al. [17] proposed a hierarchical aerial MEC framework, which optimized the computation offloading decisions of GDs by using a matching game-based algorithm and an adjustment algorithm, achieving efficient utilization of aerial computing resources and maximization of data processing. Zhang et al. [34] proposed a scheme that used UAV jammers and multi-agent deep reinforcement learning methods, where HAP was used for data training, and improved the efficiency and convergence of the algorithm by using an attention mechanism. However, the above work considered UAVs as relays and did not take into account that when the available computing resources of the UAV reached the upper limit, it would not be able to serve the GDs. In our study, GDs can offload tasks directly to the HAP without going through the UAV.

VII. CONCLUSION

In this paper, we study the problem of task offloading and resource allocation in an aerial-based MEC system. Our goal is to minimize the GD's energy consumption while guaranteeing the performance and satisfying the constraints of offloading resource constraints. We utilize stochastic optimization techniques to transform our original problem into two subproblems, namely, local computation resource allocation and offloading resource allocation. We address local computation resource allocation using a convex optimization approach. Then, we use game theory to model the competition for offloading resources among GDs and design the DGMS and TPA algorithms. Finally, the distributed DOTORA algorithm is proposed, and the theoretical analysis proof is given. Extensive experiments verify that DOTORA can perform well in terms of ensuring performance and reducing device energy consumption.

REFERENCES

- [1] H. Jiang, X. Dai, Z. Xiao, and A. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4000–4015, Jul. 2023.
- [2] Y. Cheng, J. Lu, D. Niyato, B. Lyu, M. Xu, and S. Zhu, "Performance analysis and power allocation for covert mobile edge computing with RIS-aided NOMA," *IEEE Trans. Mobile Comput.*, to be published, doi: [10.1109/TMC.2023.3302413](https://doi.org/10.1109/TMC.2023.3302413).
- [3] X. Cao, G. Zhu, J. Xu, and S. Cui, "Transmission power control for over-the-air federated averaging at network edge," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 5, pp. 1571–1586, May 2022.
- [4] F. Liu, J. Huang, and X. Wang, "Joint task offloading and resource allocation for device-edge-cloud collaboration with subtask dependencies," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3027–3039, Jul.-Sep. 2023.
- [5] S. Wang et al., "A cloud-guided feature extraction approach for image retrieval in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 292–305, Feb. 2021.
- [6] A. Asheralieva, D. Niyato, and Y. Miyana, "Efficient dynamic distributed resource slicing in 6G multi-access edge computing networks with online ADMM and message passing graph neural networks," *IEEE Trans. Mobile Comput.*, to be published, doi: [10.1109/TMC.2023.3262514](https://doi.org/10.1109/TMC.2023.3262514).
- [7] F. Pervez, A. Sultana, C. Yang, and L. Zhao, "Energy and latency efficient joint communication and computation optimization in a multi-UAV assisted MEC network," *IEEE Trans. Wireless Commun.*, to be published, doi: [10.1109/TWC.2023.3291692](https://doi.org/10.1109/TWC.2023.3291692).
- [8] Y. Sun, J. Xu, and S. Cui, "User association and resource allocation for MEC-enabled IoT networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 10, pp. 8051–8062, Oct. 2022.
- [9] T. Zhang, Y. Xu, J. Loo, D. Yang, and L. Xiao, "Joint computation and communication design for UAV-assisted mobile edge computing in IoT," *IEEE Trans. Ind. Inform.*, vol. 16, no. 8, pp. 5505–5516, Aug. 2020.
- [10] Z. Yang, C. Pan, K. Wang, and M. Shikh-Bahaei, "Energy efficient resource allocation in UAV-enabled mobile edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 9, pp. 4576–4589, Sep. 2019.
- [11] M. Dai et al., "Latency minimization oriented hybrid offshore and aerial-based multi-access computation offloading for marine communication networks," *IEEE Trans. Commun.*, vol. 71, no. 11, pp. 6482–6498, Nov. 2023.
- [12] Z. Dai, C. H. Liu, R. Han, G. Wang, K. K. Leung, and J. Tang, "Delay-sensitive energy-efficient UAV crowdsensing by deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2038–2052, Apr. 2023.
- [13] J. Ji, L. Cai, K. Zhu, and D. Niyato, "Decoupled association with rate splitting multiple access in UAV-assisted cellular networks using multi-agent deep reinforcement learning," *IEEE Trans. Mobile Comput.*, to be published, doi: [10.1109/TMC.2023.3256404](https://doi.org/10.1109/TMC.2023.3256404).
- [14] Z. Yang et al., "AI-Driven UAV-NOMA-MEC in next generation wireless networks," *IEEE Wireless Commun.*, vol. 28, no. 5, pp. 66–73, Oct. 2021.
- [15] Z. Jia, M. Sheng, J. Li, D. Zhou, and Z. Han, "Joint data collection and transmission in 6G aerial access networks," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 1–6.
- [16] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative UAV resource allocation and task offloading in hierarchical aerial computing systems: A MAPPO based approach," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10497–10509, Jun. 2023.
- [17] Z. Jia, Q. Wu, C. Dong, C. Yuen, and Z. Han, "Hierarchical aerial computing for Internet of Things via cooperation of HAPs and UAVs," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5676–5688, Apr. 2023.
- [18] Y. Gong, H. Yao, D. Wu, W. Yuan, T. Dong, and F. R. Yu, "Computation offloading for rechargeable users in space-air-ground networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 3, pp. 3805–3818, Mar. 2023.
- [19] H. Liao, Z. Zhou, X. Zhao, and Y. Wang, "Learning-based queue-aware task offloading and resource allocation for space-air-ground-integrated power IoT," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5250–5263, Apr. 2021.
- [20] P. Lai et al., "Online user and power allocation in dynamic NOMA-Based mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6676–6689, Nov. 2023.
- [21] Y. Chen et al., "Distributed task offloading and resource purchasing in NOMA-enabled mobile edge computing: Hierarchical game theoretical approaches," *ACM Trans. Embedded Comput. Syst.*, 2023, doi: [10.1145/3597023](https://doi.org/10.1145/3597023).
- [22] S. Fu, X. Feng, A. Sultana, and L. Zhao, "Joint power allocation and 3D deployment for UAV-BSS: A game theory based deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, to be published, doi: [10.1109/TWC.2023.3281812](https://doi.org/10.1109/TWC.2023.3281812).
- [23] Y. Chen, J. Zhao, Y. Wu, J. Huang, and X. Shen, "QoE-aware decentralized task offloading and resource allocation for end-edge-cloud systems: A game-theoretical approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 769–784, Jan. 2024.
- [24] N. Waqar, S. A. Hassan, A. Mahmood, K. Dev, D.-T. Do, and M. Gidlund, "Computation offloading and resource allocation in MEC-Enabled integrated aerial-terrestrial vehicular networks: A reinforcement learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21478–21491, Nov. 2022.
- [25] C. Ding, J.-B. Wang, H. Zhang, M. Lin, and G. Y. Li, "Joint optimization of transmission and computation resources for satellite and high altitude platform assisted edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 2, pp. 1362–1377, Feb. 2022.
- [26] Q. He et al., "A game-theoretical approach for user allocation in edge computing environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 3, pp. 515–529, Mar. 2020.
- [27] Z. Luo and A. Huang, "Joint game theory and greedy optimization scheme of computation offloading for UAV-aided network," in *Proc. 31st Int. Telecommun. Netw. Appl. Conf.*, 2021, pp. 198–203.
- [28] S. Jeong, O. Simeone, and J. Kang, "Mobile edge computing via a UAV-Mounted cloudlet: Optimization of bit allocation and path planning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 3, pp. 2049–2063, Mar. 2018.
- [29] Y. Dai, B. Lin, Y. Che, and L. Lyu, "UAV-assisted data offloading for smart container in offshore maritime communications," *China Commun.*, vol. 19, no. 1, pp. 153–165, 2022.

- [30] A. M. Seid, G. O. Boateng, B. Mareri, G. Sun, and W. Jiang, "Multi-agent DRL for task offloading and resource allocation in Multi-UAV enabled IoT edge network," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4531–4547, Dec. 2021.
- [31] S. Wang et al., "Federated learning for task and resource allocation in wireless high-altitude balloon networks," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17460–17475, Dec. 2021.
- [32] M. Ke et al., "An edge computing paradigm for massive IoT connectivity over high-altitude platform networks," *IEEE Wireless Commun.*, vol. 28, no. 5, pp. 102–109, Oct. 2021.
- [33] Q. Ren, O. Abbasi, G. K. Kurt, H. Yanikomeroglu, and J. Chen, "Caching and computation offloading in high altitude platform station (HAPS) assisted intelligent transportation systems," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9010–9024, Nov. 2022.
- [34] Y. Zhang, Z. Mou, F. Gao, J. Jiang, R. Ding, and Z. Han, "UAV-enabled secure communications by multi-agent deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 11599–11611, Oct. 2020.



Ying Chen (Senior Member, IEEE) received the PhD degree in computer science and technology from Tsinghua University, Beijing, China, in 2017. She is currently working toward the PhD degree with the University of Waterloo, Waterloo, ON, Canada from 2016 to 2017. She is a professor with the Computer School, Beijing Information Science and Technology University, Beijing. Her current research interests include Internet of Things, mobile edge computing, wireless net works and communications, machine learning, etc. She is the recipient of the Best Paper

Award with IEEE SmartIoT 2019, the 2016 Google PhD Fellowship Award, and the 2014 Google Anita Borg Award, 2022 Outstanding Contribution Award in 18th EAI CollaborateCom, respectively. She serves/served the leading guest editor of Springer JCC, TPC member of IEEE HPCC, and PC member of IEEE Cloud, CollaborateCom, IEEE CPSCoM, CSS, etc. She is also the Reviewer of several journals, such as the *IEEE Wireless Communications Magazine*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Internet of Things Journal*, *IEEE Transactions on Computers*, *IEEE Transactions on Cloud Computing*, and *IEEE Transactions on Services Computing*.



Kaixin Li is currently working toward the MEng degree in computer science and technology, with the Beijing Information Science and Technology University, China. His current research interests include edge computing, stochastic optimization theory, and game theory.



Yuan Wu (Senior Member, IEEE) received the PhD degree in electronic and computer engineering from the Hong Kong University of Science and Technology, in 2010. He is currently an associate professor with the State Key Laboratory of Internet of Things for Smart City, University of Macau, Macao, China, and also with the Department of Computer and Information Science, University of Macau. His research interests include resource management for wireless networks, green communications and computing, edge computing and edge intelligence, and energy informatics. He received the Best Paper Award from the IEEE ICC'2016, IEEE TCGCC'2017, IWCMC'2021, and WCNC'2023. He serves/served as the Track/Symposium Co-Chair for IEEE VTC'2017-Fall, VTC'2021-Spring, VTC'2022-Spring, ICC'2023, and GLOBECOM'2024. He is currently on the editorial board of *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Network Science and Engineering*, and *IEEE Internet of Things Journal*.



Jiwei Huang (Senior Member, IEEE) received the BEng and PhD degrees in computer science and technology from Tsinghua University, Beijing, China, in 2009 and 2014, respectively. He was a visiting Scholar with the Georgia Institute of Technology, GA, USA. He is currently a professor and the vice dean of College of Information Science and Engineering/College of Artificial Intelligence, China University of Petroleum, Beijing, and the director of the Beijing Key Laboratory of Petroleum Data Mining. He has authored or coauthored one book and more than 60 articles in international journals and conference proceedings, including *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Vehicular Technology*, *IEEE Internet of Things Journal*, *ACM SIGMETRICS*, *IEEE ICWS*, etc. His research interests include Internet of Things, edge computing, services computing, etc. He is currently on the Editorial Board of Chinese Journal of Electronics and Scientific Programming, and served as TPC members of IEEE ICWS, CollaborateCom, PRICAI, etc. He is a senior member of CCF.



Lian Zhao (Fellow, IEEE) received the PhD degree from the Department of Electrical and Computer Engineering (ELCE), University of Waterloo, Canada, in 2002. She joined the Department of Electrical and Computer Engineering at Toronto Metropolitan University (formerly Ryerson University), Canada, in 2003. Her research interests are in the areas of wireless communications, resource management, mobile edge computing, caching and communications, and IoV networks. She has been an IEEE Communication Society (ComSoc) and IEEE Vehicular Technology Society (VTS) Distinguished Lecturer (DL); received the Best Land Transportation Paper Award from IEEE Vehicular Technology Society, in 2016, Top 15 Editor Award, in 2016 for IEEE Transaction on Vehicular Technology, Best Paper Award from the 2013 International Conference on Wireless Communications and Signal Processing (WCSP), and the Canada Foundation for Innovation (CFI) New Opportunity Research Award in 2005. She has been serving as an editor for *IEEE Transactions on Wireless Communications*, *IEEE Internet of Things Journal*, and *IEEE Transactions on Vehicular Technology* (2013–2021).